Introduction

The purpose of this project is to design a generalized deep learning algorithm that can accurately segment cell types in multiplex immunofluorescence images. The idea is that eventually, it will help in improving our understanding of cell types, especially of complex cell types such as amoeboid immune cells and neurons.

Thus, with the above idea in mind, I decided to focus my efforts on macrophages, a type of cell that is notoriously difficult to accurately segment because of their amoeboid morphology. In other words, macrophages do not have the nice, rounded morphology that we see in lymphocytes and epithelial cells.

This report explains the computational pipeline that I have developed to transform raw multiplex imaging data of macrophages into something that resembles a usable data set, where millions of different types of macrophages are eventually isolated, filtered, and segmented from an image of a large tissue section. Along the way, I have incorporated parallel image processing, automated quality control using both rule-based filters and a simple CNN, and eventually some segmentation using Cellpose. I have tried to build a pipeline that tries to optimize computational bottlenecks, although I must admit that some of the steps work better than others. The final steps involve some basic validation to see if the features that we eventually extract make any biological sense.

Methods

I will go through the pipeline that I have developed in the order that it is executed. Explaining details and algorithms along the way.

Cropping

The script essentially extracts image patches of specific immune cell populations from multiplex IF images. This is one of the important steps before any spatial analysis or classifier training based on morphology is performed. The biological groundwork has already been laid in the CSV files created by the CELESTA tool. Each row in the file corresponds to a cell with coordinates and a final cell type classification such as "PD1+ CD68+ Macrophage." So essentially, we know exactly what we are looking for and where to find it.

The script then selects the cell types that we actually require and extracts clean and enhanced images around these cells.

The selection of channels is critical. For each cell type, we have specific channels that correspond to the markers used to identify that particular cell type. But importantly, we have all six channels for each and every cell regardless of their classification. This is because markers such as PD1, PD L1, CD68, and CD163 are not mutually exclusive. So, a cell classified as "PD1+ CD68+ macrophage" could also have CD163 or PD L1 expression. The original dataset was comprised of around 20 channels, and the key was to select the channels which best represented cell morphology and marker localization. In the end, we chose six channels: DAPI for nucleus localization in channel 0, PD1, CD163, PD-L1, CD68, and NaKATPase for membrane localization in channel 5. This provides us with nucleus shape, membrane boundaries, and all four functional markers for each cell, regardless of what combination of markers defined its initial cell type. Getting the channel numbers right is critical because if we map the wrong ones, we're looking at the wrong biology.

Image processing was standard with some adjustments for this specific dataset. A 50x50 pixel window is defined around each cell's center. In this case, background subtraction using a Gaussian blur removes the effects of non-uniform illumination. Intensity normalization between the 1st and 99th percentile prevents outliers from dominating the results. Contrast Limited Adaptive Histogram Equalization enhances contrast without amplifying noise. This ensures subtle marker localization is easily seen and remains consistent across images. The output is saved to a directory tree based on dataset and cell type. This makes it very easy and straightforward to pull all PD1+ macrophages for spatial proximity analysis or feed them into a deep learning network.

Indexing

This script is for me to create an index over the entire image dataset and to store the data into a SQLite database. This database is used as a central index that separates the traversal of the file system from the actual image processing. It is slow to traverse a directory tree with millions of files, and doing this traversal repeatedly will bring everything to a halt. This script does the slow traversal once, storing away information such as the relative path of the OME TIFF, the case name, and the type of macrophage. Later, other scripts like filter.py can use this database to obtain a list of file paths without ever accessing the file system until they need to access the actual image. This index also allows for resumable image processing, as other scripts can determine which images have been processed before by looking at the filtered_images table. SQLite is used here with bulk insert optimizations turned on to make the initial index creation fast enough to handle hundreds of thousands of files in a reasonable amount of time.

Initial Screening

The filter script uses quality control on the cell image patches, checking them against certain criteria. Each metric targets a specific aspect of image quality. Nucleus SNR ensures the DAPI signal is strong enough for segmentation, Laplacian variance catches blurry images, centroid offset and nucleus area fraction confirm the cell is whole and centered, membrane signal checks for expected marker expression, and marker signal requires at least one functional marker to be detectable.

The performance design eliminates the problem of multiple threads accessing the same directory on a Windows machine. This occurs because the NTFS file system locks the directory, thereby slowing the script. Instead of dividing the images between the different threads, the script divides the images based on the directory. This ensures that no two threads are accessing the same directory. Therefore, the problem of the NTFS file system locking the directory is eliminated. A dedicated writer thread with its own database connection handles batched inserts through a bounded queue, preventing the workers from being blocked on I/O while maintaining backpressure.

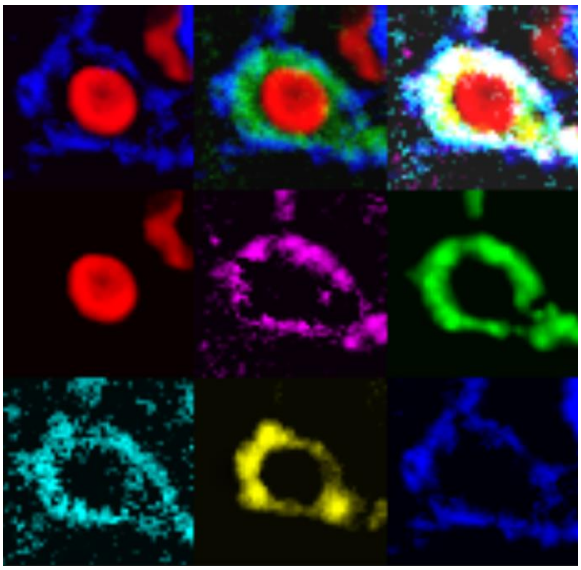
The directory groups are sorted largest-first before assignment, which balances the load and keeps total runtime close to the slowest worker rather than degrading with thread count.

Manual inspection and classification

This inspection script provides an additional critical human validation step. While quality metric scores can be carefully optimized, there's simply no substitute for visually verifying each candidate image to ensure it's truly usable for further processing. I manually went through and chose around 1500 images in total, striving for a balanced selection across the various types of macrophages. The selection criteria were very simple. If I look at the composite view of the nucleus, membrane, and marker images and cannot visually discern where the cell boundaries are, then I discard the image. This provides an additional level of verification to catch unusual nucleus shapes and membrane staining that might prevent accurate boundary drawing in the future.

The membrane marker on channel 5, is good but not perfect on its own. In some cases, the membrane staining might not be strong, but the functional markers on channels 1-4 can actually reveal the cell's shape through the expression of these markers. Having all these channels available to me when I'm inspecting the cells means I can superimpose the different markers and actually get a good idea of the cell's true morphology. A cell might have good expression of the PD1 marker, revealing the cell's shape, even if the membrane marker is weak. This is perfectly legitimate and should be included.

One of the interesting biological realities that I had to consider when working with the cells was that the different markers are not mutually exclusive. A cell that was classified as a "PD-L1+ CD68+ macrophage" might not actually exclude the presence of the PD1 marker. When I'm actually inspecting the cells, I'm not classifying them based on the presence of the different markers, I'm judging the quality of the image and whether the image appears to represent a legitimate, segmentable cell. If the image appears to have artifacts such as debris and significant noise, I exclude the image. However, some level of noise that doesn't interfere with my visual identification of the cell is acceptable, as most images are not perfect.



Red is nucleus, blue is membrane, the rest are markers. From top to bottom, left to right: 1. Nucleus+Membrane. 2. Nucleus+Membrane+Markers averaged (green). 3. All channels each with different colour overlayed. 4. Nucleus Only. 5. PD-1 (pink). 6. CD-163 (green). 7. PD-L1 (cyan). 8. CD-68 (yellow). 9. Membrane Only.

Train CNN model for quality classification

After examining about 1500 images, I had a small but reliable set of ground truth to what a "good" or "bad" cell image looked like. The filter gave me about 1.6 million cells that passed the rule-based test, but I'm certainly not going to look through all of those images by hand. The obvious solution is to train a CNN that can learn what I learned while examining images by hand and apply that to the rest of the images.

The network accepts the full 6-channel image, rather than just the features extracted by the filter script, allowing it to learn about spatial relationships that aren't captured by simple metrics, such as two cells overlapping, a cell extending past the edge of the image, out-of-focus blur that's different from in-focus blur on the background, strange features in specific areas of the image, and so forth. The network is kept simple, with three convolutional layers and adaptive pooling, to allow it to operate on the 50x50 image size. Focal loss with smoothing helps to deal with the extreme class imbalance (much more bad images than good) and ensures that the network does not overconfidently predict on potentially mislabelled data.

The training process employs a weighted sampler to ensure that the rare good class has more representation without over-sampling to the point of overfitting. Validation uses F-beta with beta set to 0.5 to monitor the performance, which means that precision is four times more important than recall. This is because when the model is run on 1.6 million cells, a false positive is worse than a false negative. I'd rather discard some potentially good cells than allow bad ones to affect the final set. After training, a threshold sweep is performed to find the operating point that satisfies the target precision (default 90%) while maximizing recall, and the best model checkpoint is saved along with this threshold to be used in the next step.

The actual threshold I have chosen is 85%, where the precision for “good” is at 90%, the F0.5 is at 0.608. The percentage of cells kept with this threshold will be theoretically around 4%. (Greatly reducing the number of cells to work with. Of course, a better classifier here can increase this and improve the result, and this should be a place for improvement later)

CNN classification and Cellpose Segmentation

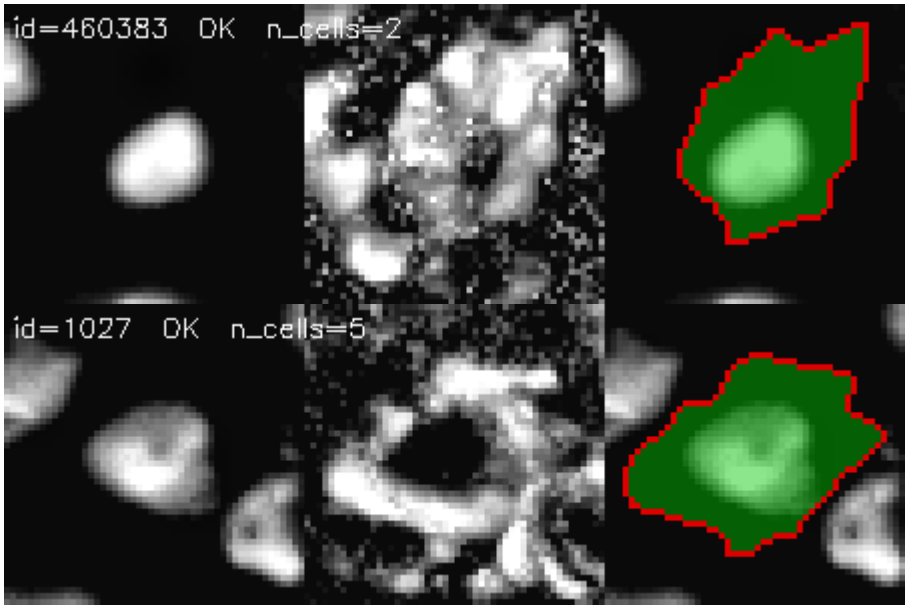
This script processes the remaining 1.6 million cells after the initial filter and uses the CNN quality classifier. Finally, it uses Cellpose for segmentation on each image classified as "good" by the CNN classifier. The main design decision here was to put these two steps together in a single pass over each image file once. This was necessary because of the sheer number of files to process, over a million, stored on an NTFS file system. In this case, the cost of disk I/O far exceeds the cost of computation. Running classification and segmentation in two separate scripts would require scanning the directory tree twice, which is impractical.

This script uses the CNN classifier to classify each image. This script uses the same CNN model trained earlier with its threshold tuned for 90% precision. Images classified as "good" are buffered by directory and passed to a separate worker process for segmentation using Cellpose. Cellpose was used off-the-shelf with the cyto3 model, without fine-tuning on this specific dataset. I have manually inspected some of the segmentation results beforehand. While it's not perfect, it's good enough for this stage of the project. However, segmentation can get tricky with ambiguous cell boundaries and unusual cell shapes. Fine-tuning Cellpose was another step I skipped due to time constraints. If I were to improve this pipeline in the future, I think the next step would be to train a custom Cellpose model on manually labelled images of this dataset.

The channel composition logic deals with the situation that not all of the marker channels are usable. It does this by checking all of the channels, which are the membrane in channel 5 and the functional markers in channels 1 through 4, for noise by checking if the largest connected component after otsu thresholding is greater than or equal to 3% of the image. Channels that are clean are then max projected with the clean membrane channel to create a clean image of the cytoplasm, which is then used for Cellpose. In the event that all of the marker channels are noisy, the function falls back to the nuclei-only version. After segmentation, if Cellpose finds multiple cells in the image, a rule picks the one closest to the centre within 20% of the image diagonal. This ensures that we're actually looking at the cell we think we are, and not a neighbor that's wandered into the frame.

The output is written to three locations. First, the 'classified_images' table is updated with the CNN prediction and confidence for all of the images. For images that were successfully segmented, the 'segmented_images' table is updated with information about where they were placed in the HDF5 stacks, and the 'cell_stats' table is updated with the morphology and the mean and standard deviation of all six channels. The HDF5 stacks are arranged by case and macrophage type, and all of the datasets are resizable to accommodate additional images as they are processed. Each HDF5 contains the normalised images, the uint16 masks, and the image IDs.

The two-phase parallelisation method caters to directories of vastly different sizes. Small directories are exclusively processed by each worker, similar to the optimisation in the filter script to prevent contention with NTFS lock. For the tiny minority of directories that are enormous in size, containing more than 80,000 images, they are processed one by one while their rows are distributed across all workers. This ensures that the GPU is never starved. A single writer thread is used to handle all database inserts and writes to HDF5 files using bounded queues, preventing workers from blocking on I/O operations.



Examples of final segmentation. Left image is only nucleus, middle image is all other channels being max projected. Right is segmentation mask.

Analysis

The analysis scripts offer a comprehensive validation of the whole pipeline by asking if the cell types we have been working with actually separate in feature space. Segmentation of more than a million cells and extraction of morphology and intensity features are a big step, and we should take a step back and ask if "PD1+ CD68+ Macrophage" and "PD-L1+ CD163+ Macrophage" are actually separable based on their physical properties and expression of markers.

The feature analysis script uses the cell_stats table in the database and feeds this through a full exploratory pipeline. It creates new features such as aspect ratio, marker sums, and coefficients of variation, and then uses PCA, t-SNE, and UMAP to visualize how these different macrophage cell types separate. It uses a Random Forest classifier to see how well these cell types can be predicted using morphology and intensity information alone. It also uses feature importance plots to see which measurements are driving this separation. It uses a Kruskal-Wallis test to see which of these features are significantly different between the cell types after correcting for multiple testing. It provides a correlation matrix, a dendrogram from a hierarchical clustering, and a heatmap of marker positivity, which shows what percentage of cells in a particular type are above the Otsu threshold for channels 1 through 4. This provides a quick way to visualize if the pipeline has produced sensible biological results.

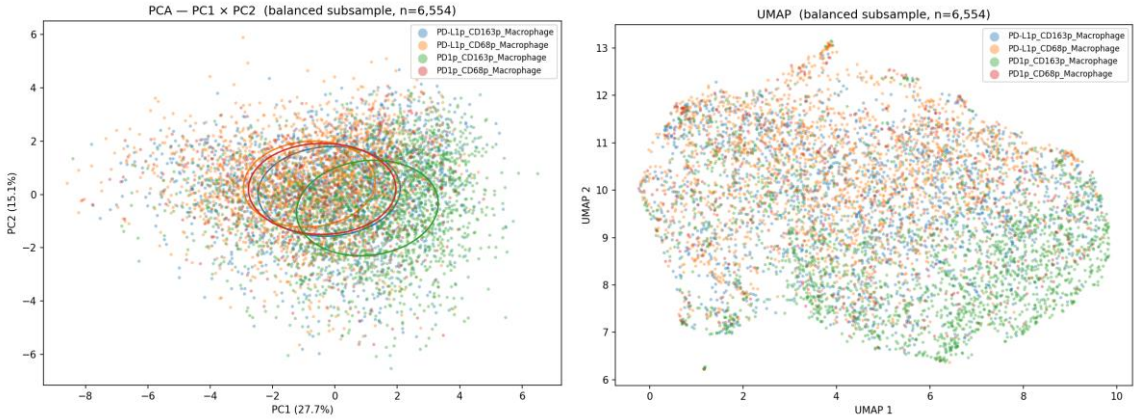
The CNN embedding analysis script takes a different approach by using a ResNet-18 network, adapted to use 6-channel images, to retrieve 512-dimensional feature vectors from the raw cell images. The weights of a network pre-trained on ImageNet are used, but their average is taken to account for the additional channels, and the network is not fine-tuned on my data. The hope is that spatial and textural features will be extracted that the morphological features miss. If the CNN embeddings do a better job of separating the cell types, it's a good indication that there's real biological signal in the images that my simple measurements aren't picking up.

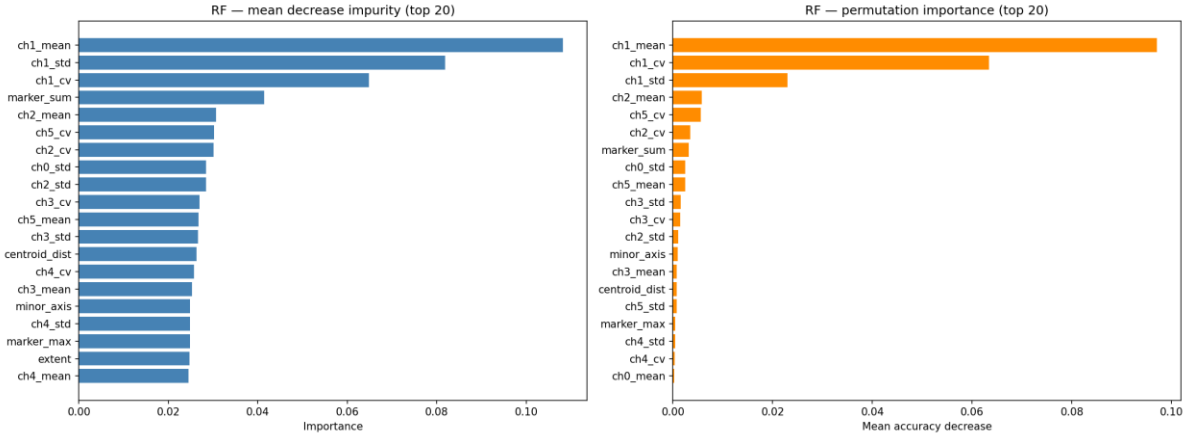
Results

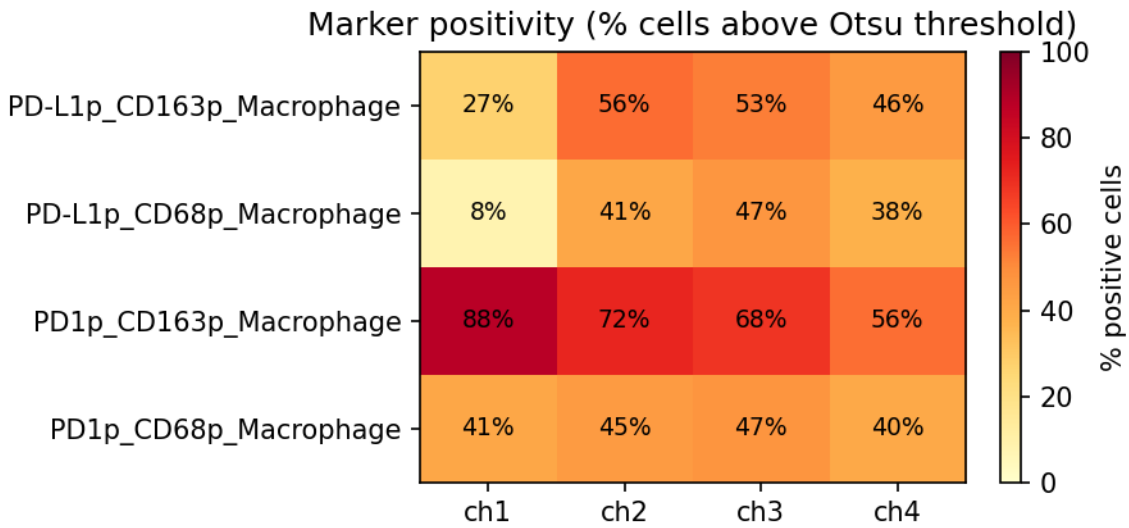
Feature Based

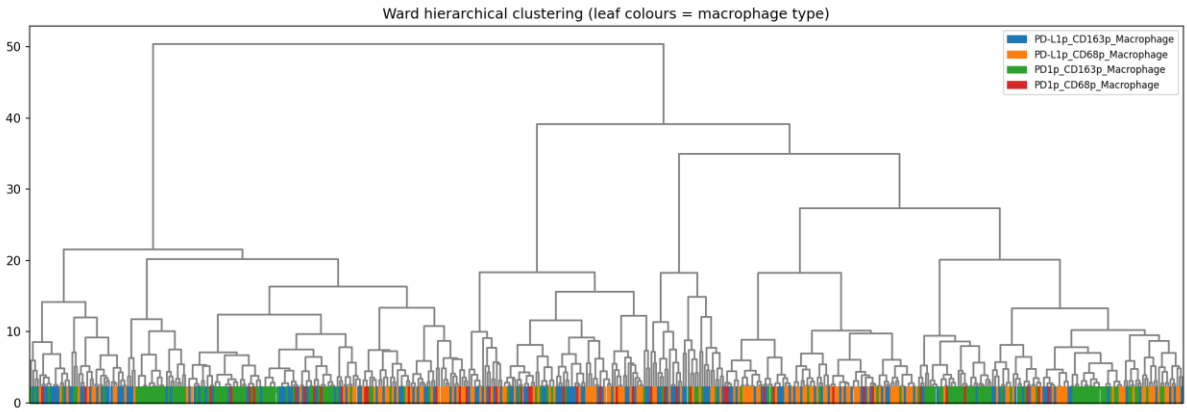
The feature-based analysis of 32,724 cells of the four types of macrophages indicates a complex scenario, as all the PCA, t-SNE, and UMAP plots indicate a single large cluster, while the cell types do not appear to be separate groups, although some internal structure is observed. The PD-L1+ CD68+ and PD1+ CD163+ types of macrophages appear to separate towards opposite ends of the clusters, while the other two types of macrophages, namely PD-L1+ CD163+ and PD1+ CD68+, appear to be distributed in the same area of the clusters. This indicates a partial but incomplete separation based on the morphology and intensity features used in the images. The Random Forest classifier results in an accuracy of 66% with five-fold cross-validation, but the balanced accuracy is reduced to 49.5%. The adjusted Rand index of hierarchical clustering is nearly zero, with a value of 0.021, indicating that the clusters do not reflect any of the cell types.

The top features driving separation are largely intensity-based rather than morphology-based. Channel 1 mean, standard deviation, and coefficient of variation are common across RF importance and Kruskal-Wallis rank feature lists, as well as marker sum and channel 2 features. There is a lack of morphology-based features such as perimeter and major axis length in the top feature lists, implying that marker expression levels are more indicative of separation between these cell types than cell morphology. The Kruskal-Wallis test found all 32 features to be significantly different between cell types after correction for multiple testing. However, the effect sizes of these differences must be small given the poor classification accuracy. This implies that the macrophage subtypes defined by CELESTA are real cell types but are very similar. This is evidenced by the visualisations, which indicate that the cell types are not randomly distributed but overlap significantly. This is also evidenced by the 66% accuracy of the classifier but the zero ARI. This makes sense biologically. These are all macrophages and will overlap in marker expression. They are not different cell types with inherently different morphology. It also implies that the separation between cell types is quantitative rather than qualitative. This is evidenced by the lack of morphology-based features in the top feature lists.









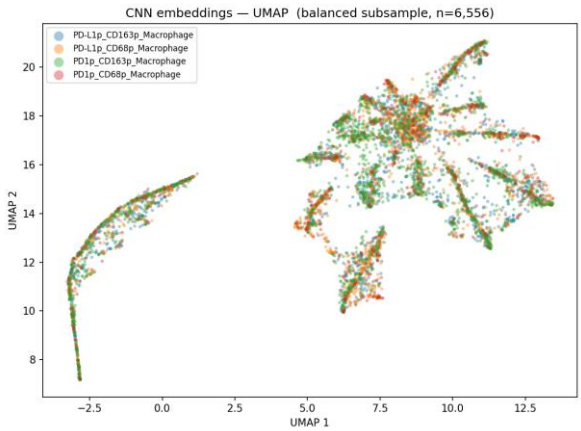
Embedding-Based

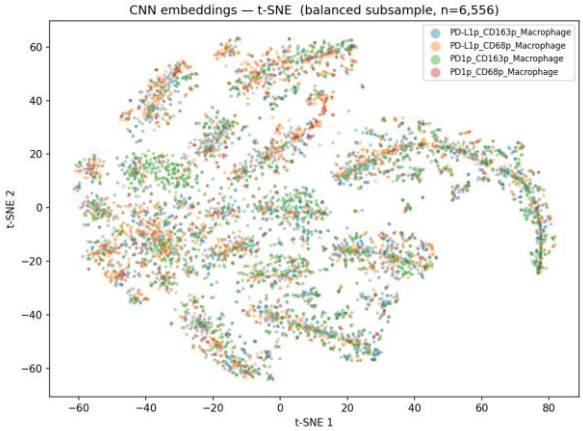
However, the CNN embedding analysis on the same 32,396 cells shows a different picture than the feature-based approach. The Random Forest classifier accuracy is 56.4%, with a balanced accuracy of 42.4%, which is much worse than the feature-based approach. The adjusted Rand index is also close to zero at -0.009. Once again, the unsupervised clustering approach does not capture the true clusters. Yet the plots show something the feature-based approach failed to capture at all. The PCA plot shows an L-shape with a sharp elbow at zero. This indicates that the embedding space is not a continuous space but has a bifurcation effect at some point.

But the UMAP plot is the most interesting. Instead of a single cluster with no distinct patterns, the UMAP shows a dense circular region that seems to have more PD-L1+ CD68+ macrophages. Then

there are the linear "rays" of points emanating from the main cluster. These rays contain a large proportion of PD1+ CD163+ cells. But one ray is much longer than the rest, creating a distinct arm in the embedding space. PD-L1+ CD163+ and PD1+ CD68+ cells are interspersed throughout the space but also show a preference for certain paths. This was not visible at all with the feature-based UMAP approach, where only a single cluster was visible.

What this likely means is that the CNN is detecting some form of spatial and textural patterns that my hand-crafted features didn’t capture. The radial pattern indicates that there might be some form of continuous variation in the cell morphology or the localization of the markers that relates to the classification labels. The dense cluster of PD-L1+ CD68+ cells might represent a more homogeneous population, while the PD1+ CD163+ cells are more diverse and spread out. However, the classification accuracy being lower than the feature-based RF wasn’t really a surprise. The CNN embeddings are 512-dimensional and never saw the task of distinguishing between these classes. The fact that the structure appears at all on the UMAP, despite not fine-tuning the model, indicates that the pre-trained model recognizes patterns that are biologically meaningful and relate to the CELESTA classification. A model that was fine-tuned on this dataset would likely perform much better, but even this off-the-shelf embedding indicates that the cell types are not random and that the process was able to capture meaningful biological variation.

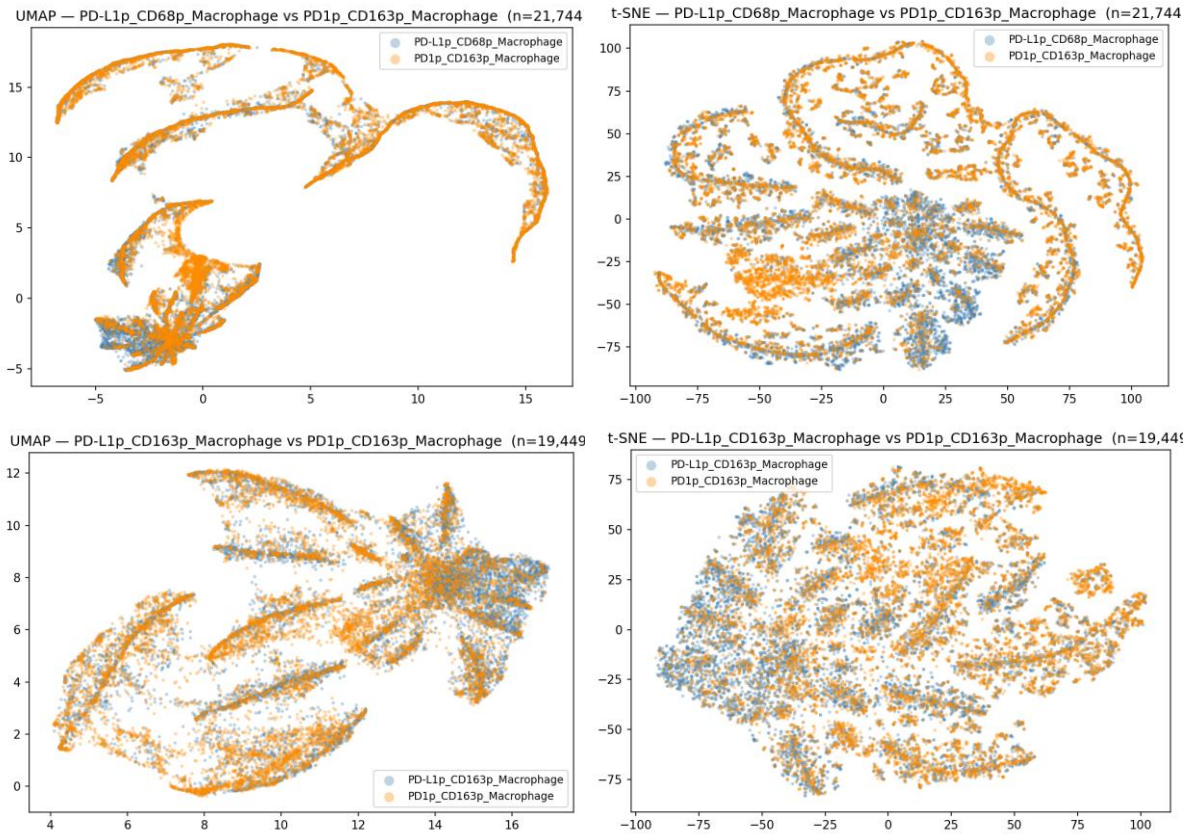




Pairwise Feature and Embedding

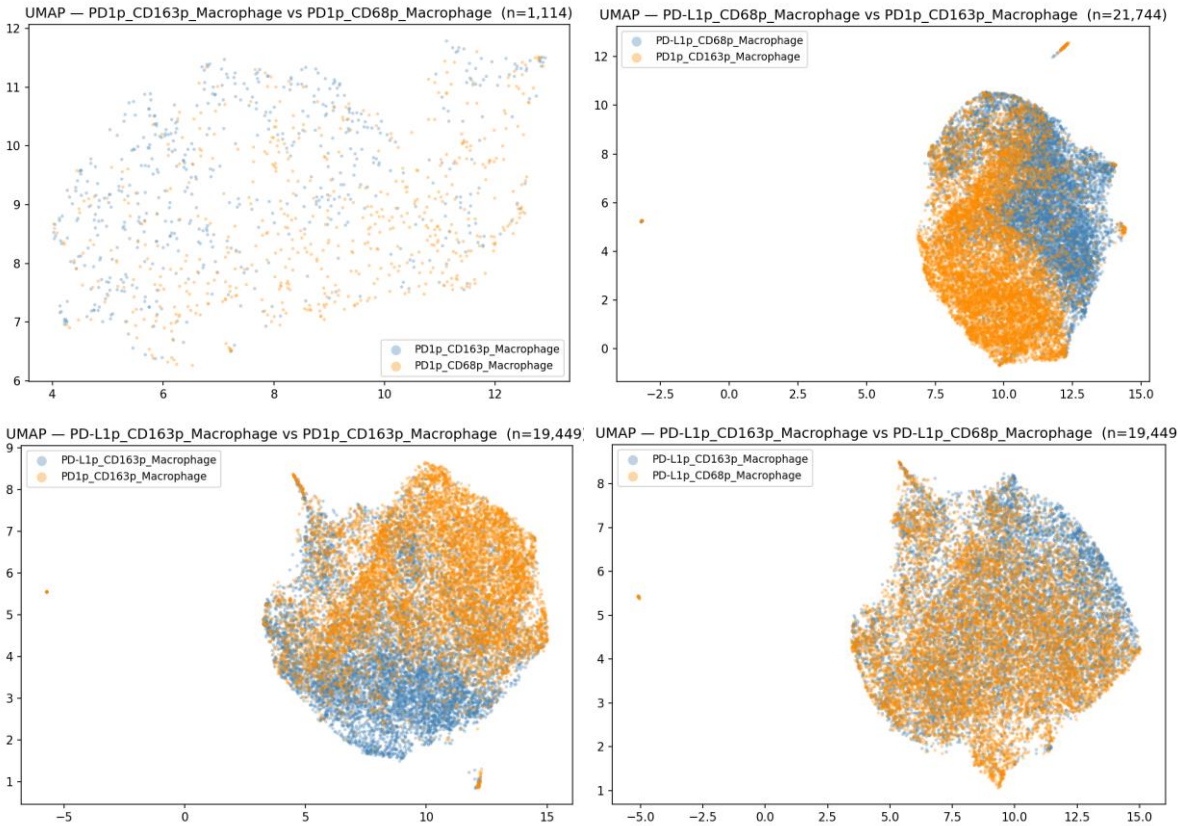
Examining the pairwise comparisons help to give a slightly different perspective. If I look specifically at the comparison of PD-L1+ CD68+ versus PD1+ CD163+ embedding, it's much clearer what's going on in the data. It's a denser area of PD-L1+ CD68+, and the PD1+ CD163+ cells make up the rays extending out from the center of that distribution. The same is true if I look at the comparison of PD-L1+ CD163+ versus PD1+ CD163+, in which case the denser area is now PD-L1+ CD163+. It seems to be that the dense core is a distinguishing feature of the PD-L1+ cells, regardless of whether they express CD163 or CD68, while the PD1+ CD163+ cells make up the outer areas of the distribution.

This all seems to indicate that the CNN is picking up on some basic difference in appearance between the cells that express PD-L1 versus the cells that express PD1, while the CD163+ versus CD68+ distinction is a secondary one that's layered on top of that basic one.



The feature-based pairwise visualisations display a different type of structure. Here, we don't see cores and rays; instead, we see a cluster divided into two halves. One half contains cells of one type, and the other half contains cells of the other type. This is precisely the kind of structure we would see if we were dealing with cell types that make a difference in quantitative rather than categorical variables. This structure is smooth and not sharply divided. This is also confirmed by the fact that we see different pairwise visualisations. This implies that each certain pairwise population is distinguishable to a certain extent; however, the boundaries are blurred.

Overall, the results suggest that the CNN and scalar features are picking up on different aspects of the biology. The scalar features are picking up on the smooth changes in marker expression levels that can support a certain level of classification. The CNN is picking up on something much more structural, possibly related to texture and spatial arrangement of the markers in the cell. The fact that PD-L1+ cells cluster together much more compactly and PD1+ CD163+ cells spread out across the embedding suggests that PD1+ cells are possibly much more morphologically diverse. Alternatively, the PD1 marker may simply be correlated with other structural features. Whatever the case, we can see that the pipeline has preserved enough of the biology to see separation in both types of visualisations, even if we do not see classification.



Conclusion

This pipeline is roughly complete, but it is also important to be realistic about how well I succeeded and where I failed. My basic goal was to take this large and messy imaging dataset and turn it into something useful for further analysis. Here, I was partially successful. My extraction and filtering scripts handled this scale reasonably well and was able to segment cleanly for a large number of cells. However, the results of the final analysis tell a more complex story, and there are a variety of reasons to consider this pipeline a proof of concept rather than a production-ready pipeline. Technical variation across different staining runs and acquisition sessions is almost certainly baked into the data. Even with normalization in my processing steps, my CNN results and feature measurements are likely to be capturing this effect in addition to any biological effect. The radial structure in our UMAP could potentially capture this effect, and the tightness of certain clusters could simply mean that these were overrepresented in certain batches.

The quality of the raw dataset itself presented many challenges. Manual inspection of the images in the labelling step revealed the object in question wasn't really a proper cell. Artifacts, debris, and ambiguous objects made it through the automated filter. While the segmentation step does remove images that cannot be segmented into a clean cell, the ones that are left are still subject to the limitations of the original acquisition.

The training of the classifier also has clear limitations. For example, with only around 1,500 manually classified images, the CNN has a relatively small amount of information to draw upon. Furthermore, the class imbalance is extreme, with far more "bad" cells than "good." While the trade-off of precision for recall minimizes the risk of false positives, this also means that potentially good cells are likely being thrown away. The entire pipeline would benefit from more careful tuning, larger and more balanced training sets, and rigorous cross-validation.

The analysis of the four different types of macrophages is certainly intriguing but is limited. While the feature-based classifier is able to obtain a high accuracy of 66% and the CNN is able to obtain a good embedding, the almost zero adjusted Rand index shows that unsupervised clustering does not

recover these groups. While the Kruskal-Wallis tests show that all of the features are indeed statistically significant, this is expected due to the small sample size.

What this is really indicating is a working infrastructure that has room to be perfected. This pipeline works well from images to segmented cells to feature measurements and visualisations. While the biological patterns that are being identified are certainly intriguing, they are not conclusive. With better experimental design, batch handling, and training sets, this same framework could potentially be used to obtain more interpretable results. But for now, the conclusions remain tentative, and the pipeline itself should be viewed as a solid foundation requiring further development rather than a finished product.